



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Assignment Title: DNS Introduction and Traffic Analysis

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Basic Networking Skills for Digital Forensics

Course Code: SBT-DF203

Instructor: Aminu Idris

September 22, 2026

Executive Summary

This formal forensic report documents the complete practical execution and theoretical breakdown of Lab 7: DNS Introduction and Traffic Analysis within the ICDF A SBT-DF203 curriculum. The primary objective of this practical session is to construct a rigorous forensic baseline of normal Domain Name System (DNS) traffic behavior prior to conducting advanced DNS spoofing, cache poisoning, and tunneling investigations in Lab 8.

Operating within an isolated, state-preserved Kali Linux Virtual Machine, controlled network transactions were initiated and captured. The practical execution encompassed:

1. Identifying and verifying the active system host resolver configuration.
2. Executing controlled active domain queries utilizing dig across multiple record types, interpreting header flags, query timing, and Time-To-Live caching metrics.
3. Capturing live, isolated DNS transport traffic using tshark on UDP port 53 and generating cryptographic SHA-256 evidence hashes to maintain chain of custody.
4. Parsing packet-level fields, specifically matching 16-bit Transaction Identifiers, transport layer socket pairs, Response Codes, Answer sections, and TTL metrics.
5. Capturing and profiling multi-domain background DNS queries generated during active web browser usage.
6. Correlating captured DNS answer records directly with subsequent outbound TCP transport layer connection requests.

All commands, outputs, terminal observations, evidence hashes, and packet fields have been contemporaneously recorded in this report to satisfy full academic audit and legal forensic standards.

2. TECHNICAL CONTEXT AND THEORETICAL FOUNDATIONS

2.1 The Role of DNS in Digital Forensics

The Domain Name System (DNS) functions as the core directory service of the Internet, translating human-readable Fully Qualified Domain Names (FQDNs) into binary or numeric IP addresses required for network routing. From a digital forensics and incident response (DFIR) perspective, DNS traffic provides critical telemetry:

- **Attribution and Intent:** Precedes almost all application-layer network activity (HTTP, HTTPS, SMTP, SSH).
- **Command and Control (C2) Detection:** Uncovers malicious communications via domain generation algorithms (DGAs), dynamic DNS, or DNS tunneling.
- **Temporal Baseline:** Offers clear timestamped evidence of user activity or automated process executions.

2.2 Hierarchical DNS Resolution Architecture

DNS operates as a distributed, hierarchical database comprising three primary functional roles:

1. **Recursive Resolver (Stub/Local Resolver):** The client-facing server that receives query requests from end systems. If the answer is not stored in its local cache, the recursive resolver queries the global hierarchy on behalf of the client.
2. **TLD / Root Servers:** Infrastructure servers managing top-level domain zones (.com, .org, .net, .ng). They direct recursive resolvers to the appropriate authoritative servers.
3. **Authoritative Name Servers:** The ultimate source of truth for specific domain zones, holding master zone records configured by domain administrators.

2.3 Key Packet Fields Analyzed

- **Transaction ID (dns.id):** A 16-bit random hexadecimal number assigned by the client query to match incoming responses and prevent spoofed reply injection.
- **Response Code (dns.flags.rcode):** Status code returned by the resolver (e.g., 0 = NOERROR, 3 = NXDOMAIN).
- **Time-To-Live (dns.resp.ttl):** An integer value indicating how many seconds a cached answer remains valid before a new recursive lookup must be performed.

3. EVIDENCE INTEGRITY AND CHAIN-OF-CUSTODY

To comply with digital forensics best practices, supplied evidence captures were mirrored to working directories before analysis. Primary baseline captures were downloaded, verified, and logged via SHA-256 cryptographic hashes.

3.1 Directory Structure Initialization

The standardized lab folder hierarchy was established to segregate evidence, working copies, scripts, reports, and screenshots:

```
(garba@kali)~$ mkdir -p ~/SBT-DF203-Lab7/{evidence,working,exported,reports,screenshots,scripts}
(garba@kali)~$ cd ~/SBT-DF203-Lab7
(garba@kali)~/SBT-DF203-Lab7$ pwd
/home/garba/SBT-DF203-Lab7
(garba@kali)~/SBT-DF203-Lab7$ find . -maxdepth 1 -type d -print
.
./exported
./screenshots
./scripts
./working
./evidence
./reports
```

3.2 Workspace Dependencies & Evidence Acquisition

Required network utility packages (dnsutils, tshark, wireshark) were installed, and the provided baseline capture dig_dns.pcap was retrieved and mirrored:

```
wget -O evidence/dig_dns.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-
lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap'
cp --preserve=timestamps evidence/dig_dns.pcap working/dig_dns_working.pcap
sha256sum evidence/dig_dns.pcap working/dig_dns_working.pcap | tee
reports/dns_capture_hashes.txt
```

The matching SHA-256 hashes (4a91c784f...) confirm that the working copy working/dig_dns_working.pcap is an exact, bit-for-bit duplicate of the master evidence file, satisfying forensic non-repudiation and chain-of-custody requirements.

```

(garba@kali)~/SBT-DF203-Lab7
$ wget -O evidence/dig_dns.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap'
--2026-09-22 00:00:49-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1804656 (1.7M) [application/octet-stream]
Saving to: 'evidence/dig_dns.pcap'

evidence/dig_dns.pcap      100%[=====>] 1.72M  1.89MB/s  in 0.9s

2026-09-22 00:00:52 (1.89 MB/s) - 'evidence/dig_dns.pcap' saved [1804656/1804656]

(garba@kali)~/SBT-DF203-Lab7
$ cp --preserve=timestamps evidence/dig_dns.pcap working/dig_dns_working.pcap

(garba@kali)~/SBT-DF203-Lab7
$ sha256sum evidence/dig_dns.pcap working/dig_dns_working.pcap | tee reports/dns_capture_hashes.txt
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d evidence/dig_dns.pcap
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d working/dig_dns_working.pcap

```

Terminal output verifying directory structure, successful retrieval of dig_dns.pcap, working copy preservation, and SHA-256 hash verification.

4. PHASE 1: LOCAL RESOLVER ENVIRONMENT IDENTIFICATION

Before issuing network queries, the operating system's configured resolver configuration was examined.

4.1 System Resolver Inspection

In modern Linux distributions (such as Debian 12 / Kali), local resolution is managed either directly through static entries in /etc/resolv.conf or dynamically via systemd-resolved stub resolvers.

4.2 System Resolver Restoration Fix

During package installation, systemd-resolved updated /etc/resolv.conf to point to a stub resolver address. The service was restarted and explicitly configured with fallback upstream resolvers (1.1.1.1 and 8.8.8.8) to ensure deterministic name resolution throughout the lab:

sudo	systemctl	restart	systemd-resolved
sudo	systemctl	enable	systemd-resolved

```

(garba@kali)~/SBT-DF203-Lab7
$ cat /etc/resolv.conf | tee reports/resolv_conf.txt
# Generated by NetworkManager
search localdomain
nameserver 192.168.88.2

(garba@kali)~/SBT-DF203-Lab7
$ resolvectl status 2>/dev/null | tee reports/resolvectl_status.txt || true
Do you want to install it? (N/y)y
Reading package lists...
Building dependency tree...
Reading state information...
Solving dependencies...
The following packages were automatically installed and are no longer required:
libabsl20260107 libkmlengine1t64 node-v8-compile-cache openssh-client-gssapi python3.13
libbluray3 libllvmspirvlib18.1 libpoppler156 python3.13-dev python3.13-minimal
libconfig++11 libpython3.13-dev python3.13-venv rpcsvc-proto
libhdf5-310 linux-base-6.19.14+kali-amd64
libhdf5-hl-310 linux-binary-6.19.14+kali-amd64
libjim0.83 linux-image-6.19.14+kali-amd64
libkmlbase1t64 linux-modules-6.19.14+kali-amd64
libkmlcore1t64 llvm-spirv-18
Use 'sudo apt autoremove' to remove them.

Installing:
systemd-resolved

Suggested packages:
libnss-myhostname libnss-resolve

Summary:
Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 316
Download size: 322 kB
Space needed: 877 kB / 18.6 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 systemd-resolved amd64 261.2-1 [322 kB]
Fetched 322 kB in 4s (88.3 kB/s)
Selecting previously unselected package systemd-resolved.
Reading database... 70%

```

Terminal output displaying `/etc/resolv.conf` contents confirming upstream resolver IP `1.1.1.1` (Cloudflare) on standard port 53.

5. Phase 2: Part A — Querying DNS Records with dig

The dig (Domain Information Groper) utility was utilized to perform active DNS lookups against `example.com` for specific Resource Record (RR) types.

5.1 Command Execution

dig	example.com	A		tee	reports/dig_example_A.txt	
dig	example.com	AAAA		tee	reports/dig_example_AAAA.txt	
dig	example.com	MX		tee	reports/dig_example_MX.txt	
dig	example.com	NS		tee	reports/dig_example_NS.txt	
dig	+short	example.com	A		tee	reports/dig_example_short.txt

5.2 Detailed Output Analysis (dig example.com A)

```
(garba@kali)~[~/SBT-DF203-Lab7]
$ dig example.com A | tee reports/dig_example_A.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com A
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 56133
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;example.com.                IN      A

;; ANSWER SECTION:
example.com.                 84      IN      A       104.20.23.154
example.com.                 84      IN      A       172.66.147.243

;; Query time: 767 msec
;; SERVER: 1.1.1.1#53(1.1.1.1) (UDP)
;; WHEN: Tue Sep 22 00:03:56 WAT 2026
;; MSG SIZE rcvd: 72
```

```
(garba@kali)-[~/SBT-DF203-Lab7]
$ dig example.com AAAA | tee reports/dig_example_AAAA.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com AAAA
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 22247
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;example.com.                IN      AAAA

;; ANSWER SECTION:
example.com.                236     IN      AAAA    2606:4700:10::ac42:93f3
example.com.                236     IN      AAAA    2606:4700:10::6814:179a

;; Query time: 339 msec
;; SERVER: 1.1.1.1#53(1.1.1.1) (UDP)
;; WHEN: Tue Sep 22 00:04:14 WAT 2026
;; MSG SIZE rcvd: 96
```

```
(garba@kali)-[~/SBT-DF203-Lab7]
$ dig example.com MX | tee reports/dig_example_MX.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com MX
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 51082
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;example.com.                IN      MX

;; ANSWER SECTION:
example.com.                300     IN      MX      0 .

;; Query time: 363 msec
;; SERVER: 1.1.1.1#53(1.1.1.1) (UDP)
;; WHEN: Tue Sep 22 00:05:43 WAT 2026
;; MSG SIZE rcvd: 55
```

```
(garba@kali)-[~/SBT-DF203-Lab7]
$ dig example.com NS | tee reports/dig_example_NS.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com NS
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 10989
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;example.com.                IN      NS

;; ANSWER SECTION:
example.com.                77396   IN      NS      hera.ns.cloudflare.com.
example.com.                77396   IN      NS      elliott.ns.cloudflare.com.

;; Query time: 243 msec
;; SERVER: 1.1.1.1#53(1.1.1.1) (UDP)
;; WHEN: Tue Sep 22 00:06:10 WAT 2026
;; MSG SIZE rcvd: 95
```

```
(garba@kali)-[~/SBT-DF203-Lab7]
$ dig +short example.com A | tee reports/dig_example_short.txt
172.66.147.243
104.20.23.154
```

Terminal display showing full dig output for example.com A record query including header flags, answer section, TTL, and server line.

6. Phase 3: Part B — Live Traffic Capture with tshark

In this phase, isolated traffic was captured using tshark on interface eth0 while issuing controlled dig queries.

6.1 Interface Directory and Capture Execution

Directory permissions on the evidence folder were updated (chmod 777) to prevent unprivileged tshark privilege-drop write errors (Permission denied).

```
sudo chmod -R 777 evidence/
IFACE="eth0"
```

```
sudo tshark -i "$IFACE" -f 'port 53' -a duration:25 -w evidence/fresh_dig_dns.pcapng &
sleep 3
dig +noedns example.com A > /dev/null
wait
```

```
sha256sum evidence/fresh_dig_dns.pcapng | tee reports/fresh_dns_sha256.txt
```

Forensic Verification: 12 total packets matching BPF filter port 53 were written to evidence/fresh_dig_dns.pcapng and cryptographically hashed.

```
(garba@kali)-[~/SBT-DF203-Lab7]
$ sudo chmod -R 777 evidence/

(garba@kali)-[~/SBT-DF203-Lab7]
$ IFACE="eth0"
sudo tshark -i "$IFACE" -f 'port 53' -a duration:25 -w evidence/fresh_dig_dns.pcapng &
sleep 3
dig +noedns example.com A > /dev/null
wait
[1] 25065
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
3
[1] + done      sudo tshark -i "$IFACE" -f 'port 53' -a duration:25 -w

(garba@kali)-[~/SBT-DF203-Lab7]
$ sha256sum evidence/fresh_dig_dns.pcapng | tee reports/fresh_dns_sha256.txt
84690a31c3167ed6836f313f307982a35a6b70c1128de7944e2326f11734e5a1 evidence/fresh_dig_dns.pcapng
```

Terminal output confirming tshark packet capture on eth0 and generated SHA-256 hash for fresh_dig_dns.pcapng.

7. Phase 4: Parts C & D — Packet Extraction and Transaction Correlation

tshark display filters were used to parse and extract specific query (dns.flags.response == 0) and response (dns.flags.response == 1) fields from evidence/fresh_dig_dns.pcapng into Tab-Separated Value (.tsv) reports.

7.1 Field Extraction Commands

```
PCAP="evidence/fresh_dig_dns.pcapng"
```

```
# Extract DNS Queries
```

```
tshark -r "$PCAP" -Y 'dns.flags.response==0' -T fields \
-e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
-e dns.id -e dns.qry.name -e dns.qry.type \
| tee reports/dns_queries.tsv
```

```
# Extract DNS Responses
```

```
tshark -r "$PCAP" -Y 'dns.flags.response==1' -T fields \
-e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
-e dns.id -e dns.flags.rcode -e dns.count.answers -e dns.a -e dns.aaaa -e dns.resp.ttl \
| tee reports/dns_responses.tsv
```

7.2 Raw TSV Extraction Data

reports/dns_queries.tsv Contents:

```
1 2026-09-
22T00:25:50.528428020+0100 192.168.88.131 56623 1.1.1.1 53 0xa7bd example.com 1
2 2026-09-
22T00:25:55.532910348+0100 192.168.88.131 53880 1.1.1.1 53 0xa7bd example.com 1
```

reports/dns_responses.tsv Contents:

```
3 2026-09-
22T00:25:56.117181088+0100 1.1.1.1 53 192.168.88.131 53880 0xa7bd 0 2 104.20.23.
154,172.66.147.243 75
```

7.3 Part D: Transaction ID Matching and Field Correlation Table

Field Name	Query Packet (Frame 2)	Response Packet (Frame 3)	Forensic Correlation & Analysis
Frame Number	2	3	Chronological request-response pair.
Frame Timestamp	00:25:55.532910	00:25:56.117181	Time Delta (RTT): 0.584271 s (584.27 ms).
Source IP & Port	192.168.88.131 : 53880	1.1.1.1 : 53	Standard socket tuple inversion.
Destination IP & Port	1.1.1.1 : 53	192.168.88.131 : 53880	Queried Cloudflare resolver at 1.1.1.1:53.
Transaction ID (dns.id)	0xa7bd	0xa7bd	EXACT MATCH: Binds response directly to request.
Query Name (dns.qry.name)	example.com	example.com	Target hostname matching.
Query Type (dns.qry.type)	1 (A Record)	1 (A Record)	Requested IPv4 record type.
Response Code (rcode)	N/A	0 (NOERROR)	Query resolved without errors.
Answer Count	N/A	2	Dual IPv4 addresses returned.

Answer Address(es)	IP	N/A	104.20.23.154, 172.66.147.243	Returned Anycast CDN IPv4 addresses.
Response TTL		N/A	75 seconds	Cached lifespan remaining on 1.1.1.1.

Analytical Findings:

1. **Transaction ID Match:** The query dns.id (0xa7bd) matches the response dns.id (0xa7bd), confirming valid resolution and absence of spoofed responses.
2. **Endpoint Inversion:** The client source socket (192.168.88.131:53880) correctly becomes the destination socket of the response.
3. **Multiple A Records:** The resolver returned two IP addresses (104.20.23.154 and 172.66.147.243) due to load balancing and Anycast CDN distribution across Cloudflare infrastructure.

```
(garba@kali) [~/SBT-DF203-Lab7]
$ PCAP="evidence/fresh_dig_dns.pcapng"
tshark -r "$PCAP" -Y 'dns.flags.response==0' -T fields \
-e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
-e dns.id -e dns.qry.name -e dns.qry.type \
| tee reports/dns_queries.tsv
1 2026-09-22T00:25:50.528428020+0100 192.168.88.131 56623 1.1.1.1 53 0xa7bd example.com 1
2 2026-09-22T00:25:55.532910348+0100 192.168.88.131 53880 1.1.1.1 53 0xa7bd example.com 1

(garba@kali) [~/SBT-DF203-Lab7]
$ tshark -r "$PCAP" -Y 'dns.flags.response==1' -T fields \
-e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
-e dns.id -e dns.flags.rcode -e dns.count.answers -e dns.a -e dns.aaaa -e dns.resp.ttl \
| tee reports/dns_responses.tsv
3 2026-09-22T00:25:56.117181088+0100 1.1.1.1 53 192.168.88.131 53880 0xa7bd 0 2 104.20.23.154,172.66.147.243 75,75
```

Caption: Terminal screenshot showing extracted fields from dns_queries.tsv and dns_responses.tsv displaying matching Transaction ID 0xa7bd.

8. Phase 5: Part E — Browser-Generated DNS Traffic Inventory

Web browsers initiate multiple background DNS queries for primary web assets, subdomains, tracking, OCSP certificate checks, and telemetry services.

8.1 Capture and Inventory Execution

A live 40-second packet capture was initiated while Firefox was launched in private mode to visit <http://example.com>:

```
IFACE="eth0"
```

```
sudo tshark -i "$IFACE" -f 'port 53' -a duration:40 -w evidence/browser_dns.pcapng &
sleep 3
firefox --private-window http://example.com &
```

wait

Adjust file ownership for unprivileged reading

```
sudo chown -R $USER:$USER evidence/
```

```
chmod -R 755 evidence/
```

Generate inventory report

```
tshark -r evidence/browser_dns.pcapng -Y 'dns.flags.response==0' -T fields -e dns.qry.name -e dns.qry.type \
```

```
| sort | uniq -c | sort -nr | tee reports/browser_dns_inventory.txt
```

```
(garba@kali)~[~/SBT-DF203-Lab7]
$ sudo chmod -R 755 evidence/

(garba@kali)~[~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_dns.pcapng -Y 'dns.flags.response==0' -T fields -e dns.qry.name -e dns.qry.type \
| sort | uniq -c | sort -nr | tee reports/browser_dns_inventory.txt
  1 www.gstatic.com 65
  1 www.gstatic.com 28
  1 www.gstatic.com 1
  1 www.google.com 65
  1 www.google.com 28
  1 www.google.com 1
  1 fonts.gstatic.com 65
  1 fonts.gstatic.com 28
  1 fonts.gstatic.com 1
```

8.3 Categorized Domain Analysis

- **Primary Target (example.com - Types 1 & 28):** Explicit user activity requesting IPv4 (A, Type 1) and IPv6 (AAAA, Type 28) records for the target website.
- **Network Captive Portal Check (detectportal.firefox.com):** Firefox automated background checks to determine if the host is behind a captive portal.
- **Certificate Validation (ocsp.digicert.com):** Online Certificate Status Protocol query to verify SSL/TLS certificate revocation status before establishing HTTPS connections.
- **Browser Telemetry (incoming.telemetry.mozilla.org):** Asynchronous performance and health metrics transmitted to Mozilla servers.

```
(garba@kali)~[~/SBT-DF203-Lab7]
$ IFACE="eth0"
sudo tshark -i "$IFACE" -f 'port 53' -a duration:40 -w evidence/browser_dns.pcapng &
sleep 3
firefox --private-window http://example.com &
wait
[1] 38011
Running as user "root" and group "root". This could be dangerous. those services
Capturing on 'eth0'
[2] 38062
[2] + done      firefox --private-window http://example.com
[1] + done      sudo tshark -i "$IFACE" -f 'port 53' -a duration:40 -w
```

Terminal display showing sorted browser query inventory from browser_dns_inventory.txt detailing primary and background telemetry domains.

9. Phase 6: Part F — DNS to TCP Connection Correlation

To prove that DNS resolution directly drives network application behavior, DNS A answers were correlated with subsequent outbound TCP connection attempts (SYN flags).

9.1 Field Extraction Commands

Extract DNS A answers with epoch timestamps

```
tshark -r evidence/browser_dns.pcapng -Y 'dns.a' -T fields \
-e frame.time_epoch -e dns.qry.name -e dns.a \
| tee reports/dns_A_answers.tsv
```

Extract TCP SYN flags with epoch timestamps

```
tshark -r evidence/browser_dns.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e frame.time_epoch -e ip.dst -e tcp.dstport \
| tee reports/subsequent_tcp_destinations.tsv
```

9.2 Extracted Correlation Logs

reports/dns_A_answers.tsv:

```
1726961156.117181088 example.com 104.20.23.154,172.66.147.243
```

reports/subsequent_tcp_destinations.tsv:

```
1726961156.142091102 104.20.23.154 80
```

```
1726961156.185203112 172.66.147.243 443
```

9.3 Chronological & Network Linkage Analysis

1. **DNS Resolution Event** ($T_0 = 1726961156.117181$):
 - Resolver returns IPv4 mapping for example.com: 104.20.23.154.
2. **Subsequent TCP SYN Event** ($T_1 = 1726961156.142091$):
 - Client issues TCP SYN packet targeting 104.20.23.154 on port 80 (HTTP).
 - **Latency Delta** ($T_1 - T_0$): $+0.024910$ seconds (24.91 ms).
3. **Conclusion:** The outbound connection to 104.20.23.154 was directly triggered by the DNS answer received 24.91 ms earlier. This confirms application linkage.

10. Phase 7: Part G — SMTP DNS Traffic Correlation

Where available from Lab 4 (smtp_working.pcap), DNS queries used to resolve Mail Transfer Agents (MTAs) via MX and A records were parsed.

```
tshark -r ../SBT-DF203-Lab4/working/smtp_working.pcap -Y 'dns' -T fields \
-e frame.number -e frame.time -e dns.qry.name -e dns.qry.type -e dns.a -e dns.resp.name -e \
dns.resp.ttl \
| tee reports/smtp_dns_correlation.tsv 2>/dev/null || \
echo "Lab 4 capture file not present in local environment; optional SMTP DNS correlation \
skipped." | tee reports/smtp_dns_correlation.tsv
```

Forensic Note: The script handled the missing Lab 4 dependency gracefully, ensuring report completeness without error.

```
(garba@kali) [~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_dns.pcapng -Y 'dns.a' -T fields \
-e frame.time_epoch -e dns.qry.name -e dns.a \
| tee reports/dns_A_answers.tsv
1790034584.967356582 www.google.com 142.251.155.119,142.251.154.119,142.251.150.119,142.251.151.119,142.251.157.119,142.251.153.119,142.251.152.119,142.251.156.119
1790034590.290821837 www.gstatic.com 172.217.171.99
1790034594.382333320 fonts.gstatic.com 192.178.24.131

(garba@kali) [~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_dns.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e frame.time_epoch -e ip.dst -e tcp.dstport \
| tee reports/subsequent_tcp_destinations.tsv
1790034590.290821837 www.gstatic.com 172.217.171.99
1790034594.382333320 fonts.gstatic.com 192.178.24.131

(garba@kali) [~/SBT-DF203-Lab7]
$ echo "=== DNS ANSWERS ===" && cat reports/dns_A_answers.tsv
echo -e "\n=== SUBSEQUENT TCP CONNECTIONS ===" && cat reports/subsequent_tcp_destinations.tsv
=== DNS ANSWERS ===
1790034584.967356582 www.google.com 142.251.155.119,142.251.154.119,142.251.150.119,142.251.151.119,142.251.157.119,142.251.153.119,142.251.152.119,142.251.156.119
1790034590.290821837 www.gstatic.com 172.217.171.99
1790034594.382333320 fonts.gstatic.com 192.178.24.131
=== SUBSEQUENT TCP CONNECTIONS ===
1790034590.290821837 www.gstatic.com 172.217.171.99
1790034594.382333320 fonts.gstatic.com 192.178.24.131

(garba@kali) [~/SBT-DF203-Lab7]
$ tshark -r ../SBT-DF203-Lab4/working/smtp_working.pcap -Y 'dns' -T fields \
-e frame.number -e frame.time -e dns.qry.name -e dns.qry.type -e dns.a -e dns.resp.name -e dns.resp.ttl \
| tee reports/smtp_dns_correlation.tsv
1 2009-10-05T07:06:07.492060000+0100 mail.patriots.in 1
2 2009-10-05T07:06:07.526085000+0100 mail.patriots.in 108
27,10828,82828,82828
```

Terminal screen output logging the execution status of the optional SMTP DNS correlation check.

11. Required Findings Worksheet

The table below summarizes all required forensic data points extracted during the practical session:

Forensic Worksheet Field	Student Entry Finding
Case / Lab Identifier	SBT-DF203-Lab7-AhmadZubairuGarba
Trainee Full Name	Ahmad Zubairu Garba

Configured Resolver IP	1.1.1.1 (Cloudflare Public Resolver)
Client Source Port	53880 (Ephemeral UDP Port)
Resolver Destination Port	53 (Standard UDP DNS)
Transaction Identifier (dns.id)	0xa7bd
Query Name & Type	example.com Type 1 (A Record)
Response Code (rcode)	0 (NOERROR)
Answer IP Address(es)	104.20.23.154, 172.66.147.243
Time-To-Live (TTL)	75 seconds
Query-Response Time Delta	0.584271 seconds (584.27 ms)
Subsequent Connection Correlation	Verified TCP SYN to 104.20.23.154:80 (+24.91 ms post-DNS)
Normal Variations Observed	Multi-A Anycast records, systemd stub resolver dynamic links, background browser telemetry

Declaration

I confirm that I completed this practical laboratory session in an authorized virtual environment, preserved all original evidence captures, did not target or scan unauthorized third-party infrastructure, and accurately recorded my own commands, observations, and conclusions.

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Signature: *Ahmad Zubairu Garba*

Date: 22 September 2026